

ANÁLISIS DEL RETO 1

Juan Sebastián Suárez Santos, 202610848, j.suarezs2@uniandes.edu.co

Samuel Rodríguez Romero, 202610831, s.rodriguez23456712@uniandes.edu.co

Nicolas Gómez Ayala, 202024568, nm.gomeza1@Uniandes.edu.co

Requerimiento <<n>>

Plantilla para el documentar y analizar cada uno de los requerimientos.

Descripción

Breve descripción de como abordaron la implementación del requerimiento

Entrada	Parámetros necesarios para resolver el requerimiento.
Salidas	Respuesta esperada del algoritmo.
Implementado (Sí/No)	Si se implementó y quien lo hizo.

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Paso 1	$O(\dots)$
Paso 2	$O(\dots)$
Paso	$O(\dots)$
TOTAL	$O(\dots)$

Análisis

Análisis de resultados de la implementación, tener cuenta las pruebas realizadas y el analisis de complejidad.

Requerimiento Ejemplo

Descripción

```
def get_data(data_structs, id):  
    """  
    Retorna un dato a partir de su ID  
    """  
    pos_data = lt.isPresent(data_structs["data"], id)  
    if pos_data > 0:  
        data = lt.getElement(data_structs["data"], pos_data)  
        return data  
    return None
```

Este requerimiento se encarga de retornar un dato de una lista dado su ID. Lo primero que hace es verificar si el elemento existe. Dado el caso que exista, retorna su posición, lo busca en la lista y lo retorna. De lo contrario, retorna None.

Entrada	Estructuras de datos del modelo, ID.
Salidas	El elemento con el ID dado, si no existe se retorna None
Implementado (Sí/No)	Si. Implementado por Juan Andrés Ariza

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

Pasos	Complejidad
Buscar si el elemento existe (isPresent)	$O(n)$
Obtener el elemento (getElement)	$O(1)$
TOTAL	$O(n)$

Análisis

A pesar de que obtener un elemento en un *ArrayList*, dada su posición, tiene complejidad constante, la implementación de este requerimiento tiene un orden lineal $O(n)$. Esto debido a que, lo primero que se hace es verificar si el elemento hace parte de la lista. Específicamente, a la hora de buscar un elemento en una lista, en el peor de los casos es necesario recorrer toda la lista, es decir, complejidad lineal.

REQUERIMIENTO 2

Descripción: Este requerimiento busca los pedidos cuyo precio por caja (Price_per_Box) está dentro de un rango dado por el usuario (un precio mínimo y un precio máximo). Para resolverlo se recorre una sola vez la lista de pedidos (guardada en un Array List), y por cada pedido se revisa si su precio está dentro del rango. Si el pedido cumple, se cuenta, se van sumando sus valores para sacar los promedios de descuento, inversión en mercadeo y precio por caja, y se compara ese pedido contra el mejor que se llevaba guardado hasta el momento para quedarse con: el pedido más reciente, el de mayor monto (Amount) y el de menor monto. Cuando hay un empate en la fecha más reciente, gana el pedido de mayor monto; cuando hay un

empate en el mayor o menor monto, gana el pedido con menor precio por caja, tal como lo pide el enunciado.

Entrada: Precio mínimo, precio máximo y el catálogo con los pedidos ya cargados.

Salidas: Tiempo de ejecución, cantidad de pedidos en el rango, promedios de descuento, inversión en mercadeo y precio por caja, y los datos completos del pedido más reciente, el de mayor monto y el de menor monto.

Implementado: Si

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

- Obtener el tamaño de la lista de pedidos ($al.size$) = $O(1)$
- Recorrer los n pedidos uno por uno con el for, sacando cada uno con $al.get_element$ = $O(n)$
- Por cada pedido: comparar el precio contra el rango, sumar los valores y comparar contra el mejor guardado hasta el momento = $O(1)$ por pedido

TOTAL = $O(n)$

Análisis

Aunque acceder a un elemento del Array List por su posición es de tiempo constante ($O(1)$), como no se sabe de antemano cuáles pedidos cumplen el rango de precio, es necesario revisar los n pedidos uno por uno. Por eso el algoritmo completo queda en tiempo lineal, $O(n)$. No hay ninguna forma de resolver este requerimiento sin recorrer todos los pedidos al menos una vez, porque el filtro depende de un dato (el precio) que puede variar en cualquier pedido.

REQUERIMIENTO 5

Descripción: Este requerimiento busca, dentro de un producto específico y un rango de fechas, el pedido de menor o mayor monto (Amount), según lo que pida el usuario (filtro "MENOR" o "MAYOR"). Como los pedidos están guardados en una lista enlazada (Single Linked List), no se puede acceder directamente a una posición: hay que ir recorriendo nodo por nodo desde el primero, siguiendo el apuntador al siguiente nodo, hasta que no queden más. Por cada nodo se revisa si el pedido es del producto buscado y si su fecha está entre la fecha inicial y la final. Si cumple, se cuenta, se suman sus valores para los promedios de precio, cajas enviadas e inversión en mercadeo, y se compara contra el mejor pedido que se llevaba guardado hasta el momento (el de mayor o menor monto, según el filtro), quedándose con el mejor de los dos. En caso de empate en el monto, gana el pedido con menor precio por caja.

Entrada: Catálogo con los pedidos, filtro ("MAYOR" o "MENOR"), nombre del producto, fecha inicial y fecha final.

Salidas: Tiempo de ejecución, cantidad de pedidos encontrados, promedios de precio por caja, cajas enviadas e inversión en mercadeo, y los datos del pedido resultante (precio por caja, cajas enviadas, monto, canal, fecha e inversión en mercadeo).

Implementado: Si

Análisis de complejidad

Análisis de complejidad de cada uno de los pasos del algoritmo

- Recorrer todos los nodos de la lista enlazada desde el primero hasta el último, siguiendo el apuntador "next" = $O(n)$
- Por cada nodo: revisar si el producto y la fecha cumplen el filtro = $O(1)$ por cada nodo ($O(n)$)
- Si cumple: sumar los valores para los promedios y comparar contra el mejor pedido guardado hasta el momento = $O(1)$ por cada nodo ($O(n)$)

TOTAL = $O(n)$

Análisis

A diferencia de un Array List, en una lista enlazada no existe una forma de "saltar" directamente a un nodo: siempre hay que empezar desde el primero e ir avanzando de a un nodo a la vez. Por eso, revisar todos los pedidos para encontrar los que cumplen el producto y el rango de fechas toma un tiempo lineal, $O(n)$, recorriendo en el peor de los casos los n nodos de la lista.

Nota: Antes de dar este requerimiento por terminado, revisar el caso en que no exista ningún pedido que cumpla el filtro (`cantidad_pedidos = 0`): en la versión actual, esa validación compara contra `-1` en vez de comparar contra `None`, y puede producir un error en vez de devolver "Unknown". Corregirlo en grupo antes de la entrega.